

BOILER ROOM PROJEKT

PawPals — Djuradoptionsplattform

Kurs 5: Backendutveckling i Node.js, databaser och säkerhet

PROJEKTINFORMATION	
Projektnamn:	Arbetstid:
PawPals	4 timmar Boiler Room (måndag) + onsdagar & torsdagar 09:00–11:00 (teamarbete)
Team:	
Johannes · Axel · Emelie · Dennis	
Beskrivning:	
PawPals är en djuradoptionsplattform där djurskyddsorganisationer listar djur för adoption. Adoptörer kan söka, filtrera och skicka in ansökningar som handläggs med ett statusflöde. Systemet byggs som ett fullstack-projekt med React i frontend och ett RESTful API med Node.js, Express och MongoDB i backend.	

Projektbeskrivning

PawPals är ett grupprojekt som löper parallellt med kursen. Varje sprint bygger vidare på de kursmoment som introduceras den veckan — vilket gör att projektet växer i takt med att teamet lär sig.

Projektet följer en frontend-first approach: React-frontend byggas med mock-data i de första sprinterna, och backend kopplas in successivt när kursen introducerar respektive teknik. Det gör att teamet alltid har något visuellt och körbart att visa — oavsett var i kursen vi befinner oss.

FUNKTIONELLA KRAV

PawPals ska uppfylla följande funktionella krav fördelade per område.

Djur

Djurlista

- Visa alla djur i ett grid med kort (bild, art, ras, ålder, kön)
- Filtrera på art, ras, ålder, kön och storlek
- Filtrera på boendetyp och erfarenhetsnivå (matchning)
- Sortera på senast tillagd och ålder
- Fritextsökning
- Paginering (visa X djur åt gången)
- Tomt-state när inga djur matchar filter

Djurprofilsida

- Bildgalleri med primärbild och miniatyrer
- Art, ras, ålder, kön, storlek
- Hälsostatus (vaccinerad, kastrerad etc.)
- Personlighetsbeskrivning (fritext)
- Vilken organisation som listar djuret
- Knapp: Ansök om adoption
- Knapp: Spara som favorit

Konton & autentisering

Registrering

- Välj roll vid registrering: Adoptör eller Organisation
- Fält: namn, e-post, lösenord
- GDPR-samtyckesruta (obligatorisk)
- Validering av alla fält

Inloggning

- Logga in med e-post och lösenord
- Felmeddelande vid fel uppgifter
- Håll mig inloggad (JWT)
- Logga ut

Mina sidor (Adoptör)

- Visa och redigera profil
- Se mina ansökningar med status
- Se mina favoriter
- Se mina notifikationer

Adoptionsansökan

Skicka ansökan (Adoptör)

- Formulär: boendetyp, storlek på bostad
- Formulär: erfarenhet av djur
- Formulär: familjesituation (barn, allergier)
- Formulär: motivering (fritext)
- GDPR-info om känsliga uppgifter
- Bekräftelsesida efter inskickad ansökan

Statusflöde

- Inskickad → Granskas → Godkänd / Neka
- Adoptör ser aktuell status på sina ansökningar
- Meddelande/motivering vid nekad ansökan

Organisation

Hantera djur

- Lägg upp nytt djur (alla profilmät + bilder)
- Redigera befintligt djur
- Ta bort djur (mjuk radering)
- Lista över organisationens alla djur

Hantera ansökningar

- Se alla inkomna ansökningar per djur
- Filtrera på status
- Granska ansökan (se sökandes uppgifter)
- Godkänn eller neka med valfri motivering

Favoriter & notifikationer

Favoriter

- Spara ett djur som favorit från profilsidan
- Ta bort favorit
- Lista över alla sparade favoriter

Notifikationer

- Få notis när ett bevakat djur uppdateras
- Få notis när ansökningsstatus ändras
- Markera notis som läst
- Lista över alla notifikationer

Uppföljning

- Uppföljningsformulär skickas ut efter genomförd adoption
- Frågor om hur det går för djuret
- Frågor om adoptörens upplevelse
- Organisation kan se inkomna uppföljningar per djur

MVP & NICE TO HAVE

MVP är den minsta versionen av PawPals som fungerar från start till slut. En organisation ska kunna lista ett djur och en adoptör ska kunna söka om det.

MVP — Måste finnas

Område	Krav
Konton	Registrera sig som Adoptör eller Organisation, logga in och logga ut
Djur	Organisation lägger upp ett djur (namn, art, ras, ålder, beskrivning, en bild)
Djurlista	Adoptör ser en lista med alla djur och kan klicka sig in på en profilsida
Ansökan	Adoptör skickar in en ansökan på ett djur
Handläggning	Organisation ser inkomna ansökningar och kan godkänna eller neka
Status	Adoptör ser sin ansökans nuvarande status

Nice to have — Om tid finns

Område	Funktioner
Djur & sökning	Bildgalleri, filter (art/ras/ålder), fritextsökning, paginering, matchningsfilter, redigera/ta bort djur
Favoriter & notiser	Spara favoriter, lista favoriter, notis vid uppdatering och statusändring
Uppföljning	Uppföljningsformulär efter adoption, organisation ser svar
Övrigt	Organisationsprofil, redigera användarprofil, enkel statistik-dashboard

SYSTEMKRAV

Följande krav ska uppfyllas i projektet. Kraven speglar kursinnehållet och introduceras successivt under kursens gång.

Krav	Beskrivning
RESTful API	Node.js + Express med routes/, controllers/, middleware/
TypeScript	Hela kodbasen typifierad, interfaces för alla datamodeller
MongoDB + Mongoose	Schemas, validering, relationer (populate), MongoDB Atlas
Zod-validering	Inputvalidering + centraliserad felhantering med typade felklasser
Autentisering	Lösenordshashning med bcrypt, tokenbaserad auth med JWT
RBAC	Rollbaserad åtkomstkontroll — Admin, Organisation, Adoptör
GDPR	Dataminimering, mjuk radering, dokumentation av dataflöden
Loggning	Winston strukturerad loggning utan persondata i loggar
Testning	Jest + Supertest — integrationstester för alla CRUD-endpoints
Driftsättning	Render/Railway (backend) + Netlify/Vercel (frontend), MongoDB Atlas
Dokumentation	README med endpoints, arkitekturdiagram och autentiseringsflöde
Git-workflow	Feature branches, pull requests och code reviews

Roller (RBAC)

Systemet ska ha minst tre roller med olika behörigheter:

Roll	Behörighet
Admin	Full åtkomst till hela systemet
Organisation	Lägga upp och hantera djur, granska och handlägga ansökningar
Adoptör	Söka och filtrera djur, skicka och följa ansökningar, hantera favoriter

Datamodeller

Följande Mongoose-scheman ska implementeras med TypeScript-interfaces:

Modell	Nyckeldata
User	namn, e-post, lösenord (bcrypt), roll, samtycke (GDPR), deletedAt
Organization	namn, beskrivning, kontakt, koppling till User
Animal	art, ras, ålder, kön, hälsostatus, personlighet, bilder, organisation
Application	sökande (User), djur (Animal), status, boendetyp, erfarenhet, familj
Favorite	User → Animal-relation

FollowUp	koppling till Application, formulärsvar, datum
AnimalImage	url, ordning, primärbild, koppling till Animal

VECKOPLANERING

Kursen löper över 12 veckor och är indelad i 6 sprinter à 2 veckor. Varje sprint bygger på de kursmoment som introduceras den veckan.

Vecka	Sprint	Tema	Fokus
KV 1	Sprint 1	Projektstart & React-UI	Repo & mappstruktur, Express-server, React + Vite setup, designsystem, startsida med mock-data
KV 2	Sprint 1	Projektstart & React-UI	Navbar, footer, djurkort-komponent, startsida klar, "Så fungerar adoption"-sektion
KV 3	Sprint 2	TypeScript + MongoDB & Djurlista	TypeScript konfigurerat i hela kodbasen, delade interfaces (Animal, User, Application)
KV 4	Sprint 2	TypeScript + MongoDB & Djurlista	MongoDB Atlas kopplat, Animal-schema, djurlistningssida med filter och profilsida klar
KV 5	Sprint 3	CRUD, Tester & Validering	Animal CRUD-endpoints (POST, GET, PUT, DELETE), Jest + Supertest-tester
KV 6	Sprint 3	CRUD, Tester & Validering	Zod-validering, typade felklasser, centraliserad felhantering, auth-sidor i React
KV 7	Sprint 4	Auth (JWT/bcrypt) & RBAC	POST /auth/register & /auth/login med bcrypt + JWT, auth-middleware
KV 8	Sprint 4	Auth (JWT/bcrypt) & RBAC	RBAC med tre roller (Admin, Org, Adoptör), frontend kopplas mot riktigt API
KV 9	Sprint 5	GDPR, Loggning & Driftsättning	Soft-delete, anonymisering, Winston-loggning utan persondata, Application-API
KV 10	Sprint 5	GDPR, Loggning & Driftsättning	Ansökningsflöde kopplas mot backend, backend → Render, frontend → Netlify/Vercel
KV 11	Sprint 6	Dokumentation, QA & Inlämning	README med endpoints och diagram, code review alla PRs, komplettera tester
KV 12	Sprint 6	Dokumentation, QA & Inlämning	Final QA, Helmet.js + rate-limiting, projekthinlämning torsdag, skriftlig reflektion

Arbetstider

Tillfälle	Tid & plats
Boiler Room (måndag)	4 timmar — Malmö
Teamarbete (onsdag)	09:00–11:00 — Digitalt
Teamarbete (torsdag)	09:00–11:00 — Digitalt

LEVERABLER

Vid kursens slut (vecka 12) ska följande levereras:

Kod och system

- Fullständigt REST API i Node.js + Express med TypeScript
- MongoDB Atlas-databas med Mongoose-scheman och relationer
- Autentisering med bcrypt + JWT och rollbaserad åtkomstkontroll (RBAC)
- GDPR-anpassning: soft delete, anonymisering, samtycke
- Strukturerad loggning med Winston utan persondata
- Driftsatt backend på Render/Railway och frontend på Netlify/Vercel
- React-frontend kopplad mot produktion-API

Testning

- Jest + Supertest — integrationstester för alla CRUD-endpoints
- Tester för autentisering, RBAC och ansökningsflöde
- Testdatabas med mongo-memory-server

Dokumentation

- README med alla endpoints dokumenterade med exempel
- Systemarkitekturdiagram
- Autentiseringsflöde-diagram
- GDPR-dokumentation: vilka data lagras och varför
- Körningsinstruktioner (lokalt + produktion)

Git-workflow

- Feature branches: feature/s[sprint]-[namn]
- Pull requests med code review innan merge
- Commit-prefix: feat:, fix:, docs:, test:, chore:
- Skyddad main-branch — kräver godkänd PR

Definition of Done

En issue anses klar när:

- Koden är godkänd via PR med minst en code review
- TypeScript kompilerar utan fel
- Nya backend-endpoints har minst ett integrationstest (från sprint 3)
- Ingen persondata förekommer i loggfiler
- Feature-branch är mergad till develop via PR